

CS444 A2-A4 Design Document

Name Resolution, Type Checking, and Static Analysis

Arnav Tripathi (a22tripa)

Ryan Maxin (rsmxin)

Andreja Japundzic (ajapundz)

Design and Implementation	2
Overview	2
Organization	2
Abstract Syntax Tree	3
Symbol Table	3
Global Environment	4
Trade-Offs	4
Challenges	5
Testing and Known Issues	5
Testing	5
Known Issues	6
Contributions	6
Thoughts	7

Design and Implementation

Overview

In the first assignment, we created a compiler frontend for scanning, parsing, AST building, and weeding. These modules worked on each input file individually in chronological order with no context being shared between file instances. For assignments 2-4, we now had to implement the name resolution, type checking, and static analysis, and now we had to use the context from *every* file to inform our decisions. To accomplish this we organized these assignments into a series of passes which would work on the ASTs built from our frontend. Passes will either mutate the ASTs (in the case of binding symbol ids), or add a side table which stores whatever context we discovered during that pass. Each pass is implemented as its own module, which helps maintain clear separation between different phases of compilation.

Organization

This section describes the organization of our compiler middleend, outlining the modules in the order they are executed and explaining the role each module plays in the compilation process. Each step is structured around one complete pass through all the class ASTs of the input program:

1. **Resolver Module (/lib/middlend/resolver):** This module first does a short pass through each AST output from the frontend and creates a global environment which contains all the declared classes and packages. Once this is done, the full AST pass for this step checks duplicate names and performs the full type linking step defined in the assignment. Our “symbol table” data structure was the key to holding, building, and handling this context, which we describe in more detail later.
2. **Set Builder (/lib/middlend/set_builder):** This module contains the AST pass which builds all the required sets for the hierarchy checking step (i.e. super, inherit, etc.), as well as building other unneeded but helpful sets like a set of all members and fields to allow for O(1) membership checks. This pass also verifies acyclicity and a couple other hierarchy rules which were convenient to check on the fly.
3. **Hierarchy Checker (/lib/middlend/hierarchy_checker):** With the sets built from the previous module, this step's AST pass goes through all the necessary hierarchy checks, as defined in the assignment.
4. **Disambiguating (/lib/middlend/disambiguator):** This pass makes every ambiguous name in the AST unambiguous by classifying it and rebinding it. The names are made unambiguous following the specifications in the Java spec. The main idea is that during AST building, the generic Name node from the parse tree is converted into a list of *name*, where each *name* is one of Package Name, Type Name, etc. Thus, during type linking, we can pattern match for Type Name and during disambiguation, we can pattern match for Ambiguous Name. By the end of the traversals, every Ambiguous Name in the AST has been converted to one of Package name, Type Name, etc.
5. **Type Checking (/lib/middlend/type_checker):** This pass does type checking as defined in the assignment. We have a module with a function for each rule discussed in class and the spec and then we simply do a pass through the AST, using pattern matching to call the appropriate function.

6. **Static Analysis (/lib/middlend/static_analysis):** Walks the ASTs and, for each class, runs Reachability on every constructor body and every method body that exists. `reachability.ml` does a simple forward flow check (Maybe/No) to reject unreachable statements, ensure non-void methods can't fall off the end without a return, and enforce that local variables have initializers and don't reference their own `value_sid` in that initializer. It uses `const_eval.ml` to constant-evaluate boolean while conditions so `while(true)` is treated as non-terminating and `while(false)` makes the body unreachable, and it records results in `analysis_tables.stmt_reachability`.

Abstract Syntax Tree

Our main and most detailed data structure is our AST, with our name resolution and static analysis logic centered around navigating the AST.

In terms of architecture, we have tree nodes for compilation units, classes, interfaces, members, statements, and expressions. Every name in the program is stored as a type "qname" which is a list of type "name". The actual name type is one of `PackageName`, `TypeName`, `ExpressionName`, `MethodName`, or `AmbiguousName`. The idea behind this was that after parsing, we would have a collection of qualified names in the program with detailed information on what exactly each name in the path represents. When doing name resolution logic, we can pattern match for specific names like `TypeName` and `AmbiguousName` to perform the required logic.

For nodes that require names, such as a method call, we store the name as a type "bound_name". This type stores the actual name (as a "qname" described above) and a *mutable* "sid". This mutable type represents a symbol id which we use to connect the use of a name to its declaration in the program. We assign each "bound_name" a unique "sid" during AST creation and then as we go through the name resolution logic, we connect that "sid" to the declaration using a hash table "binding_to_decl". Now when navigating the AST, we can search up the "sid" in the hash table and instantly get the declaration of the name.

For hierarchy checking we have a type "type_meta" which for each Java type in the program records: type declaration AST node, hashtable for field name to field declaration AST node, hashtable for methods, `super_set`, `candidate_set`, `replace_set`, `inherit_method_set`, and `inherit_field_set`. Each hashtable uses either the method signature as a string or the fully qualified type names as the string for the key. These data structures are then filled in using the set builder logic and used to perform all the hierarchy checks.

The type checker introduces one additional important internal data structure, which is a semantic type language with cases for primitive types, reference types, array types, null, and void. We found it easier to express assignability, castability, subtype checks, and special cases like String concatenation using an internal semantic type representation instead of the surface AST type syntax. This is particularly true once array assignability to Object, Cloneable, and Serializable, primitive widening, and null assignability all have to coexist in one consistent set of rules.

Symbol Table

In our compiler, the "symbol table" is a single global, mutable record that combines (a) a cross-file binding map (so every declaration/use has a unique identity) with (b) the per-file and per-scope machinery

needed to resolve names correctly. The heart of it is `binding_to_decl`: one global table that maps any symbol id (SID) to the corresponding declaration node in the AST.

We seed this table up front in `build_global_env` by walking all compilation units and assigning initial SIDs to each class or interface declaration. Then, as we resolve each file, we inject its `file_info` (which captures that file's imports and current type context). That lets type names resolve through `get_fqn`, and `lookup_and_bind_type` records the resolved SID directly into the AST's type-use nodes.

Value names are handled separately using a local-scope structure: `string_to_sid` maps an identifier to a stack of `value_sids`, while `undo/declared/method_bound_names` implement scope push/pop and duplicate checks. Importantly, `lookup_and_bind_var` only consults this value stack (and rejects qualified value names), so we never “fall back” to the type environment and accidentally treat a valid type name as a valid value (e.g., allowing `return A` just because `A` is a declared type).

After resolution, we package the result as a single `binded_ast` root containing the complete global `binding_to_decl` plus the per-file ASTs. We then carry `SymTable` through later passes as the shared context container by storing additional pass results inside it (for example, `binding_to_meta` from set-building and `analysis_tables` from type checking), so every pass can consistently look up `sid → decl/meta/analysis` without rebuilding context.

Global Environment

Our `lib/middleend/resolver/global_env.ml` performs the first semantic sweep over the ASTs. It builds a types table from fully qualified names to `type_info`, a packages set that includes every package prefix, and the initial `binding_to_decl` table keyed by symbol id. The package-prefix construction is important because package-versus-type ambiguity only makes sense if prefixes like `java` and `java.lang` are considered valid package names in their own right. Once all types have been collected, the pass checks that no type name conflicts with a package name. We also use the global environment later as the shared source of truth for type visibility during import resolution, disambiguation, and subtype reasoning.

One subtle but useful design choice here is that the global environment does not try to store every later semantic fact. It is intentionally limited to whole-program information that is stable and globally visible: what packages exist and what types exist. More context-sensitive information, such as what imports are active in a file or what locals are visible in a scope, is kept elsewhere. This separation kept the global environment easy to reason about and avoided turning it into an oversized structure that mixes together facts from multiple abstraction levels.

Trade-Offs

One important trade off, which we discussed a bit in class, was whether to store all the information in the AST or to maintain a whole ecosystem of side tables as we discover more information. We work on the compiler in an iterative manner, completing each assignment in a specified order. This raises a problem as the information we needed to store in the AST for assignment 2 is not sufficient for assignment 4. Yet, since our AST is immutable (OCaml implementation) we cannot easily “attach” new fields to nodes as we discover information. So the question was whether to spend time going back and updating all the pattern matching logic each time we needed more information, or to just create a new table.

We ultimately decided to create a new table every time we needed to keep track of some information. While we pay some complexity whenever later passes want information, we accept this extra

bookkeeping as it enhances our quality of life (as busy students), it plays well into OCaml's strengths, and it keeps data structures separate and focused on one concern. Overall, we are happy with the design of our compiler so far.

Another tradeoff was whether we should merge name-res + disambiguation into a single pass. This would force the compiler to answer context-sensitive questions before enough program information was made available. Instead, we split this logic into our resolver initially, allowing it to establish declarations, imports, and explicit type links, and afterwards run a disambiguation pass which would classify the ambiguous names with the accessible file and inheritance context. This meant writing more infrastructure, but allowed us to process the logic in a clean way.

Challenges

The hardest part of A2-A4 was that Java names are much more intertwined than they first appear. Imports, package prefixes, local shadowing, implicit this, inherited fields, and type visibility all influence whether the same textual identifier should be interpreted as a type or an expression. Conceptually our team had difficulty understanding the full picture of how every name gets mapped to an eventual binding. This resulted in bugs in one phase often surfacing as failures in later phases, which made debugging especially difficult early on. We addressed this by making the compiler more explicit about context. File information is stored separately from local scope information, the disambiguator runs only after enough metadata has been built, and each phase throws its own exception type so we can quickly tell what part of the pipeline rejected a program.

Testing and Known Issues

Testing

For testing, we continued the same practices from the first assignment. Each assignment had its own directory under the test_suite folder which contained the given test cases as well as any additional test cases we decided to include. To run these test cases, we have a bash script which feeds all the required files for each test case to the ./joosc executable and reports expected vs actual output, giving us a total score over all test cases.

One key feature we added to aid us in testing was a specific module (/lib/frontend/reader/reader.ml) which reads in all the inputted files. It has logic which checks that all files have a valid ".java" suffix and also ensures that duplicate files are not read in twice. It also adds support for a "@stdlib" argument which causes the reader to read the standard library files when included. This allows us to test locally with the standard input included and then easily not include it in cases like submitting to marmoset where it is already included.

When designing extra test cases for these assignments, one of our focuses was on designing test cases which are valid Joos programs. Our logic behind this was that as we add more restrictions on what a valid Joos program is, the likelihood of a valid program not being compiled due to a mistake in logic, greatly increases. Thus, we added many test cases with the goal of double checking that certain logic only rejected bad programs. We found this very helpful in refining the name resolution logic in our compiler and feel it will go a long way in passing all the private test cases.

Known Issues

As of 03/16/2026, there are no known issues with the implementation of our compiler. We have a 100% pass rate on the public test cases provided to us, and we also have included some edge cases for name resolution and static analysis in the test-suite/ directory that match the intended behaviour.

Contributions

Arnav:

- Added implementation for first 5 rules of hierarchy checking
- Created first implementation of set creation (super, candidate, replace, inherit)
- Worked on type checking/static analysis

Andreja:

- Built the global environment and worked on the local environment building algorithm with Arnav
- Built the type linking with help from Ryan
- Designed and implemented disambiguation and linking of names
- Worked on type checking and static analysis with Arnav and Ryan

Ryan:

- Built the "symbol table" data structure for scoping, added symbol ids to AST nodes, and implemented binding from uses to declarations (for both type checking and name resolution)
- Changed the middle end to process all files in one shared context so global environment building works across the full program.
- Built the final inherit and replace sets, through a DFS traversal of the classes. Checked for cyclic inheritance.
- Implemented type hierarchy checking
- Brought the stdlib and testing suite into our compiler directory and added the A2, A3, and A4 test suites.
- Added constant expression parsing, lowered for loops to while loops in the abstractor, and implemented type checking and static analysis with Arnav and Andreja

Thoughts

The transition from A1 to A4 is a strong example of why phase separation matters in compiler construction. The project did not jump directly from parsing to type checking. Instead, we inserted intermediate passes that calculated and stored program context which made later checks much easier to reason about. The middle-end took a similar amount of time to implement as the A1 assignment, since semantic analysis required stronger invariants between the phases. In fact, a substantial amount of time went into debugging between the phases, for instance, when an apparent type checking issue was occurring because of improper name resolution or inheritance metadata.

OCaml remains a good choice for our compiler. The compiler now uses many variants of the AST nodes so the strong type system helps us maintain the invariants that we define. Any potential mistake becomes a type error instead of a pesky runtime bug that we have to spend prolonged time solving. Our biggest gripe is that at times we feel like we are fighting against the functional style of the language, in that whenever we wish to change the AST (such as to add type annotations), we have to change every previous pass back down to the abstractor to ensure pattern matching works. In some cases we did do this but it felt messy overall.

From a group-work perspective, these assignments also made it clear how important it is to agree on interfaces early. A lot of the collaboration went smoothly when the boundaries between modules were explicit, such as what the resolver should guarantee before disambiguation runs or what metadata should exist before type checking begins. When those boundaries were unclear, even small changes could ripple outward and cause confusion in several files at once. In the end, one of the most useful habits we developed was to think about each phase in terms of the exact information it promises to the next phase. That mindset made the compiler easier to extend and is probably one of the biggest lessons we will carry forward from the project.